

CHAOTIC TOWER

Physics-Based Competitive Tower-Stacking Game

Nama : **Danish Al Fayyadh Sunarta**

NPM : **2406416951**



Daftar Isi

01

Gambaran Umum Proyek

Deskripsi, inspirasi, dan fitur utama

02

Game Modes

Survival, Race, dan Puzzle Mode

03

Magic Spell System

Light & Dark spells, mekanisme cooldown

04

Tech Stack & Arsitektur

Frontend, backend, REST API, database

05

Design Patterns

9 design patterns dalam implementasi

06

Tantangan & Solusi

Hambatan teknis dan cara mengatasinya

07

Future Improvements

Roadmap pengembangan ke depan

Apa itu Chaotic Tower?

Chaotic Tower adalah game **physics-based competitive tower-stacking** yang terinspirasi oleh **Tricky Towers** dan **Tetris**.

Pemain menyusun balok Tetromino setinggi dan serapi mungkin, sambil menggunakan **magic spells** untuk mempersulit lawan dalam mode multiplayer.



Inspirasi

Tricky Towers — physics stacking

Tetris — Tetromino shapes & mechanics



Platform & Distribusi

- Desktop (Windows .exe)
- Diunduh via itch.io
- Backend di-deploy di Railway
- No local setup diperlukan oleh pemain

Game Modes



Survival Mode

OBJECTIVE

Bertahan selama mungkin

Pemain memiliki 3 lives. Setiap balok yang jatuh ke luar area bermain mengurangi satu nyawa. Game berakhir ketika semua nyawa habis.

- 3 lives per pemain
- Score = jumlah balok yang berhasil ditumpuk
- Leaderboard berdasarkan max height (meter)

Race Mode

OBJECTIVE

Capai garis finish setinggi 20 meter

Speedrun siapa yang paling cepat menyusun menara hingga mencapai garis finish di ketinggian 20 meter. Tidak ada lives yang hilang karena balok jatuh.

- Target height: 20 meter
- Score = waktu tercepat (millisecond)
- Leaderboard ascending (waktu terkecil menang)

Puzzle Mode

OBJECTIVE

Tumpuk balok sepadat mungkin

Pemain harus menyusun balok di bawah garis laser. Balok yang jatuh ke lantai menurunkan batas laser. Di multiplayer, P1 dan P2 mendapat urutan balok yang identik untuk keadilan.

- Garis laser turun tiap balok jatuh
- Target: 25 balok di bawah laser
- Identical sequence untuk P1 & P2

Sistem Magic Spell

Cara Kerja: Pemain mendapatkan spell acak setiap kali menara melewati **kelipatan 4 meter**. Jenis spell disembunyikan di HUD dengan indikator "**MAGIC READY**" berwarna cyan. Setelah digunakan, ada **cooldown 15 detik** sebelum spell berikutnya tersedia.

✨ Light Spells (Self Buff)

Cement Spell

Mengubah balok terakhir menjadi StaticBody yang immovable. Balok berubah warna abu-abu.

Ivy Spell

Merekatkan dua balok teratas menggunakan Box2D WeldJoint. Balok berubah warna hijau tumbuhan.

Lightning Spell

Menghancurkan (destroy) balok terakhir yang ditempatkan untuk memperbaiki kesalahan penempatan.

🌑 Dark Spells (Opponent Debuff)

Frost Spell

Menurunkan friction balok lawan menjadi 0.02f — sangat licin. Berlangsung 15 detik.

Weight Spell

Memunculkan balok raksasa (scale 2x, massa 2x lipat) yang dapat meruntuhkan menara lawan. Berlangsung 10 detik.

Speed Up Spell

Meningkatkan kecepatan jatuh otomatis balok lawan sebanyak 3x lipat. Berlangsung 10 detik.

Teknologi yang Digunakan



Frontend (Game Client)

LibGDX

Game Engine & rendering framework (Java)

Box2D

Physics engine — rigid body simulation

LWJGL3

Desktop backend / window management

FreeType

Custom pixel font rendering

FitViewport

OpenGL viewport management



Backend (REST API Server)

Spring Boot

REST API framework (Java)

Spring Data JPA

Data layer with Hibernate ORM

PostgreSQL

Relational database (deployed on Railway)

Docker

Containerization via Dockerfile multi-stage build

Railway

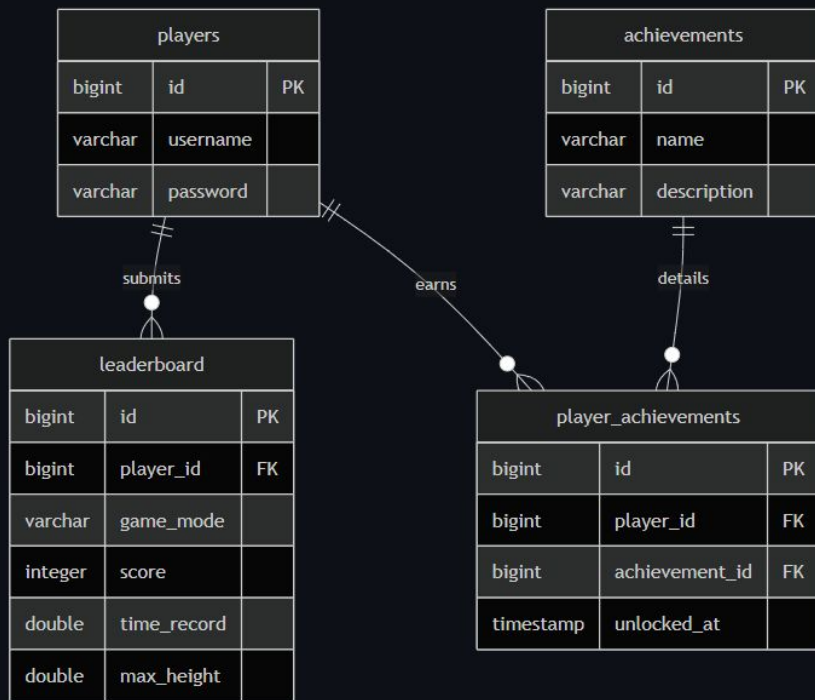
Cloud hosting platform (free tier)

REST API Endpoints

Base URL: <https://chaotictower-production.up.railway.app/api>

Method	Endpoint	Fungsi
POST	/players/login	Autentikasi login pemain
POST	/players/register	Mendaftarkan akun pemain baru
GET	/players/{id}	Mengambil detail data pemain by ID
POST	/leaderboard/submit	Submit skor & statistik game ke leaderboard
GET	/leaderboard/top10	Mengambil top 10 skor untuk suatu mode
POST	/achievements/unlock	Membuka achievement untuk pemain tertentu

Entity-Relationship Diagram (ERD)



9 Design Patterns dalam Implementasi

Game Loop Pattern

CORE ARCHITECTURE

Menggunakan LibGDX render() sebagai main loop untuk update physics dan graphics rendering secara berkala setiap frame.

📌 *Main.java → render()*

Factory Pattern

CREATIONAL

BlockFactory mengotomatisasi pembuatan 7 variasi bentuk Tetromino Block berdasarkan parameter definisi shape yang telah ditentukan.

📌 *BlockFactory.java → spawnBlock()*

Singleton Pattern

CREATIONAL

Diterapkan pada GameAssetManager & BlockFactory agar hanya ada satu instance yang mengelola asset dan pembuatan objek di seluruh lifecycle aplikasi.

📌 *GameAssetManager.getInstance() | BlockFactory.getInstance()*

Object Pool Pattern

PERFORMANCE

Pool<Block> mendaur-ulang objek Block yang sudah dihancurkan untuk menghindari memory leak dan mencegah lag akibat Garbage Collection spikes.

📌 *BlockFactory.java → Pool<Block>*

State Pattern

BEHAVIORAL

Mengatur navigasi antar screen menggunakan interface Screen LibGDX. Transisi dari MainMenuScreen → ModeSelectScreen → PlayingScreen → GameOverScreen.

📌 *Screen implementations*

9 Design Patterns — Lanjutan

Observer Pattern

BEHAVIORAL

Box2D ContactListener memantau collision antar balok melalui callback beginContact() dan mengatur perubahan fisik secara asinkron di preSolve() sebelum world physics di-solve.

📌 *PlayingScreen.java* → *setupContactListener()*

Strategy Pattern

BEHAVIORAL

Interface GameModeStrategy memisahkan logika menang/kalah dan pesan hasil akhir untuk tiga mode (Survival, Race, Puzzle) secara modular dan interchangeable.

📌 *GameModeStrategy.java* | *SurvivalStrategy* | *RaceStrategy* | *PuzzleStrategy*

Command Pattern

BEHAVIORAL

InputCommand memetakan penekanan keyboard menjadi objek command (MoveLeftCommand, MoveRightCommand, RotateCommand, SoftDropCommand) untuk mendukung local multiplayer.

📌 *InputCommand.java* | *InputHandler.java*

Repository / DAO Pattern

DATA ACCESS

Spring Data JPA interfaces (PlayerRepository, LeaderboardRepository, AchievementRepository) memisahkan data access layer dari business logic di backend server.

📌 *repository/*.java* (Backend)

Implementasi Box2D Physics



Controlled State

Saat balok aktif dikontrol pemain:

gravityScale	= 0	// tidak terpengaruh gravitasi
friction (side)	= 0	// tidak nyangkut di dinding
fixedRotation	= true	// tidak berputar saat dikontrol
linearDamping	= 0	// pergerakan responsif
movement step	0.5f unit	// pergeseran horizontal diskrit



Settled State

Saat balok mendarat dan settle:

gravityScale	= 1.2f	// gravitasi penuh aktif
friction	= 0.55f	// cengkeram balok di bawah
angularDamping	= 2.5f	// meredam rotasi berlebih
fixedRotation	= false	// boleh berputar wajar
snapping	grid snap	// posisi dikunci ke grid

Tantangan & Solusi

01 JVM Crash – Box2D Physics Modification

✗ Masalah:

Box2D melarang modifikasi body/fixture/joint di dalam ContactListener callback. Melanggar aturan ini menyebabkan corrupt memory dan crash.

✓ Solusi:

Deferred Execution Queue (blocksToSettle). Callback hanya mendaftarkan aksi ke antrian — eksekusi dilakukan setelah world.step() selesai di main loop.

02 Physics Tuning – Jitter & Ghost Collision

✗ Masalah:

Menara rawan runtuh akibat goyangan kecil (micro-nudging) atau tabrakan hantu. Balok juga sering stuck di dinding saat dimasukkan ke celah sempit.

✓ Solusi:

Hitbox dikurangi -0.01f, scale objek 0.5f, preSolve friction filter di dinding sampling, angularDamping=2.5f dan gravityScale=1.2f saat settle.

03 API Instability – Railway Cold Starts

✗ Masalah:

Koneksi API HTTP terkadang putus mendadak saat cold start pada hosting Railway gratisan, ditambah masalah CORS Policy pada desktop client.

✓ Solusi:

Exception handling yang lebih robust di ApiClient.java dan konfigurasi CORS di Spring Boot WebConfig untuk mengizinkan cross-origin request dari desktop.

04 OpenGL Buffer Crash – Screen Transition

✗ Masalah:

Menekan tombol mode sebelum SpriteBatch selesai rendering menyebabkan dispose() dipanggil saat thread render masih aktif — crash 'no buffer allocated'.

✓ Solusi:

Guard flag transitioning dan memindahkan handleInput() setelah blok batch.end() selesai dipanggil.

Game Loop & Lifecycle

On Github 

Future Improvements

Visual & Sound Effects

menambahkan sound effect saat block saling bersentuhan dan visual effect saat spell casting

Achievement

table databasenya sudah ada dan siap untuk digunakan, namun belum sempat implementasi achievements atau milestone untuk player peroleh dan UI untuk menampilkannya

Sprite-Based Block Textures

Ganti ShapeRenderer geometris dengan sprite-sheet bertekstur untuk customization tambahan sesuai keinginan player

Physics

Penyempurnaan tuning physics lebih lanjut agar lebih nyaman untuk dimainkan seperti pada game Tricky Towers

**Terima
Kasih!**

